

Mandatory Hand-In 1

Introduction to Security

Part 1 - Elgamal Private key and decryption

To decrypt the student number hidden within the encrypted message, we need to use the private key. But as we do not possess the private key, we also need to acquire the private key. To obtain the private key, we can look at the publicly known information.

$$g = 42$$

$$p = 29837$$

$$\text{public key} = 22690$$

From here we need the formula to generate a public key.

$$g^x \bmod p = \text{public key}$$

As we already know every variable except x (the private key), even the “result” of the calculation, we can guess or rather calculate what the private key is. Though it is important to notice that we mod by p , which means that we only need to guess numbers up until p , as it will just loop around to the beginning again. If we then insert all possible values between 0 and p in x 's place and see if the result is equals the public key, we then (hopefully) get a match of calculated public key and the known public key. This will then give us the correct x , i.e. the private key.

$$42^{24774} \bmod 29837 = 22690$$

To then decrypt the message using our newly obtained private key, we use the following formula.

$$c1 = 23447$$

$$c2 = 8372$$

$$s = c1^{\text{privatekey}} \bmod p = 23447^{24774} \bmod 29837 = 2805$$

$$s^{-1} = s^{p-2} \bmod p = 2805^{29837-2} \bmod 29837 = 25763$$

$$\text{decrypted} = c2 \cdot s^{-1} \bmod p = 8372 \cdot 25763 \bmod 29837 = 26000$$

This means that the decrypted student number is **26000**.

Part 2 - Modifying encrypted message

To modify the encrypted student number, we will first need the decrypted student number, where we then can use the property of the Elgamal algorithm called multiplicative homomorphic. This means that the encrypted message can be modified multiplicatively, which will be decrypted into a message where it has also been modified by the same amount. Secondly we need to find a multiplicative link between the unencrypted original student number and mine. We can do this with the following formula.

$$myStudent = 22304$$

$$\begin{aligned} ratio &= (myStudent \cdot (decrypted^{p-2} \bmod p)) \bmod p \\ &= (22304 \cdot (26000^{29837-2} \bmod 29837)) \bmod 29837 = 21814 \end{aligned}$$

Afterwards we only need to multiply the original encrypted student number with the ratio (and modulus), which will give us the correct modified encrypted message, which will decrypt into my student number.

$$modifiedEncrypted = 8372 \cdot 21814 \bmod 29837 = 24368$$

This encrypted number will be decrypted into **22304**, which is my student number.

Part 3 - Improved encryption

To improve upon the current encryption method I chose to use a TLS encryption, which uses certificates, Diffie-Hellman, AES, MAC and some other clever tricks. TLS encryption will provide confidentiality, authentication and most importantly integrity to the system. I will be using the built in Python library called “ssl” to encrypt the network traffic between client and server. Furthermore I will be generating certificates and keys using “openssl” terminal tool. Due to the requirement of no pre-shared secrets exist (i.e. certificates), I will not be authenticating the server certificate during the establishment of connection between the server and client. Although in a real world scenario, you would make use of pre-shared CA certificates, which requires agreement from a CA provider.

The process of establishing a connection on the client side is the following:

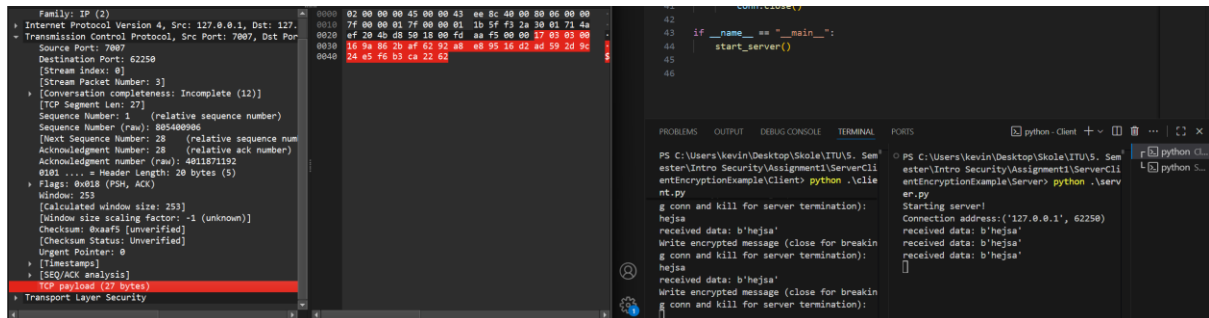
- Load the certificate and key into the Python “ssl” library context
- Create an internet socket
- Wrap that socket into a TLS encryption method with the “ssl” context
- Try and establish a connection on the specified port and IP address

The process of establishing a connection on the server side is the following:

- Load the unverified certificate and key into the Python “ssl” library context
- Create an internet socket

- Wrap that socket into a TLS encryption method with the “ssl” context
- Start listening on the specified port and IP address
- Accept the incoming connection request

To ensure that the messages are indeed encrypted between the client and server, we will look at Wireshark for a message from the specified port and IP address.



As seen above, we get a hexadecimal delivered message and if try and decode the message from hexadecimal into UTF-8 text, we will get unrecognized characters. As it is not UTF-8 anymore but encrypted.

???

That will conclude that the message is encrypted, and as it is TLS encrypted we do not have a quick way of brute forcing the decryption of the messages as a middleman.